

Recall Agent Technical Report

Technical Report: “Recall” — Free, No-API-Key AI Agent with Long-Term Memory

Source file: `Recall_Agent_Free_NoAPIKey.ipynb` **Platform:** Google Colab (T4 GPU) **Report date:** August 14, 2026

1. Purpose and Positioning

This notebook implements a conversational AI agent — text and voice — that maintains **long-term memory** across sessions, entirely using free, open-source, self-hosted components. It is explicitly framed as a cost-free substitute for a paid, API-based version of the same system:

Capability	Paid equivalent	Free version (this notebook)
Chat / reasoning	OpenAI GPT-4o	Qwen2.5-1.5B-Instruct (local)
Speech-to-text	OpenAI Whisper API	<code>openai-whisper</code> (local inference)
Text-to-speech	OpenAI TTS API	<code>edge-tts</code> (free, no account)
Memory embeddings	OpenAI embeddings API	ChromaDB’s built-in local embedding model

No API keys, billing, or external accounts are required anywhere in the pipeline. The explicit trade-off acknowledged in the notebook: a 1.5B-parameter local model is far less capable than GPT-4o, so this build is positioned as a way to validate the **memory + voice pipeline architecture**, not as a production-quality assistant.

2. System Architecture

The notebook builds a five-layer stack:

1. **Model layer** — local causal LM for generation
2. **Memory layer** — persistent vector store for long-term recall
3. **Agent layer** — orchestrates short-term + long-term context into prompts
4. **Voice layer** — STT/TTS wrappers
5. **Interface layer** — Gradio UI (text tab + voice tab)

Data flow (text chat)

```
user message
  → memory.get_context_block() (semantic search over past exchanges)
  → build system prompt (base instructions + retrieved memories)
  → append last 12 turns of session history
  → apply_chat_template → tokenize → model.generate()
  → decode reply
  → append to session history
  → memory.add_exchange() (persist this turn)
  → return reply
```

Data flow (voice chat)

```
microphone audio → Whisper transcribe() → agent.chat() → edge-tts synthesize() → playback
```

3. Component-by-Component Breakdown

3.1 Dependencies (Cell 1)

```
transformers, accelerate, chromadb, gradio, openai-whisper, edge-tts, sentencepiece
```

All installable via pip with no registration step. `accelerate` supports `device_map="auto"` for GPU placement; `sentencepiece` is required for the Qwen tokenizer.

3.2 Persistent Storage Setup (Cell 2)

Optionally mounts Google Drive so the Chroma vector DB survives runtime disconnects:

```
CHROMA_PATH = "/content/drive/MyDrive/recall_agent_free/chroma_db"
```

A commented-out fallback sets `CHROMA_PATH = "/content/chroma_db"` for ephemeral, session-only memory. This is a clean, well-documented optionality — good practice for a Colab notebook meant to be reused across sessions.

3.3 Chat Model (Cell 3)

```
MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME, torch_dtype=torch.float16, device_map="auto")
```

- Uses fp16 for a lighter memory footprint (fits comfortably on a T4's ~16 GB VRAM).
- `device_map="auto"` lets `accelerate` handle placement without manual `.to("cuda")` calls.
- ~3 GB download, cached after first run.

3.4 Long-Term Memory Layer (Cell 4)

Built on ChromaDB with its **default local embedding function** (no embedding API call):

```
client = chromadb.PersistentClient(path=CHROMA_PATH)
collection = client.get_or_create_collection(name="memory_default_user")
```

The `LongTermMemory` class exposes three methods:

Method	Role
<code>add_exchange(user, assistant)</code>	Stores each turn as a single document "User: ... \nAssistant: ..." with a UUID id and a <code>timestamp</code> metadata field
<code>search(query, k)</code>	Runs a semantic similarity query against the collection, guarding against <code>k</code> exceeding the current document count
<code>get_context_block(query, k)</code>	Wraps retrieved memories into a labeled block ready to inject into the system prompt

Design notes: - Memory is stored **per exchange** (user+assistant pair) rather than per-message — a reasonable granularity for conversational recall. - Retrieval is unconditional semantic search (no metadata filtering by user, date, or topic), which is fine for a single-user demo but would need per-user namespacing (e.g., separate collections or a `user_id` metadata filter) to scale to multiple users. - The `n = min(k, self.collection.count())` guard prevents Chroma from erroring when there are fewer stored documents than `k`.

3.5 Agent Logic (Cell 5)

```
class Agent:
    def chat(self, user_message):
        memory_block = self.memory.get_context_block(user_message, k=5)
        ...
```

Combines three context sources into every generation call: 1. **System instructions** (fixed persona/behavior prompt) 2. **Retrieved long-term memories** (top-5 semantically relevant past exchanges) 3. **Short-term session history** (last 12 turns, i.e. 6 exchanges, kept in a Python list)

Generation parameters: `max_new_tokens=300`, `temperature=0.7`, `do_sample=True`. This is a standard sampling configuration — reasonable for conversational variety, though it means outputs are non-deterministic across runs.

After each reply, the exchange is appended to both `self.history` (session memory) and `self.memory` (persistent memory) — a clean separation between short-term working memory and long-term retrievable memory, functionally similar in spirit to the RAG-style agent loop from your earlier Colab project, but simpler (single-shot retrieval + generation, no explicit plan/reflect stages).

3.6 Voice Layer (Cell 6)

- **STT:** `whisper.load_model("base")` — a mid-tier Whisper checkpoint balancing speed and accuracy (`tiny`/`small` noted as alternatives in comments).
- **TTS:** `edge-tts` using the `en-US-AriaNeural` voice, run via `asyncio.run()` inside a synchronous wrapper — necessary because `edge_tts.Communicate.save()` is async but Gradio callbacks here are called synchronously.

3.7 Gradio UI (Cell 7)

Two tabs: - **Text chat:** `gr.ChatInterface` bound directly to `agent.chat`. - **Voice chat:** a microphone `gr.Audio` component triggers `handle_voice`, which transcribes → generates a reply → synthesizes audio → auto-plays the response.

`demo.launch(share=True, debug=True)` generates a public Gradio share link, useful for accessing the Colab-hosted app from outside the notebook (e.g., on a phone), though this does expose the demo publicly for the link's lifetime.

4. Strengths

- **Zero-cost, zero-key reproducibility** — every component runs from open weights or free hosted inference (`edge-tts`), so the notebook can be re-run by anyone with a Colab account and no credentials.
- **Clean separation of concerns** — model loading, memory, agent orchestration, voice I/O, and UI are each isolated in their own cell/class, making the notebook easy to debug or extend piece by piece.
- **Sensible defaults with escape hatches** — e.g., the commented-out non-Drive path for `CHROMA_PATH`, adjustable Whisper model size, adjustable `k` for retrieval.
- **Persistent memory done right at small scale** — using UUIDs as document ids and timestamps as metadata leaves room for future filtering/pruning without needing a schema migration.

5. Notable Gaps / Risks (worth addressing before treating this as a portfolio piece)

1. **Single global collection, no user isolation.** `memory_default_user` is hardcoded — fine for a personal demo, but a multi-user deployment would leak one user's memories into another's context.

2. **No memory pruning or deduplication.** The collection grows unboundedly; near-duplicate exchanges (e.g., repeated greetings) will accumulate and dilute retrieval quality over time.
3. **No error handling around model generation or Whisper/edge-tts calls.** A network hiccup in `edge-tts` (it calls a free Microsoft endpoint, so it is not fully offline despite being “free/no-key”) will raise an unhandled exception in the Gradio callback.
4. **edge-tts is not truly local.** Worth noting in documentation: despite the “no API key” framing, `edge-tts` streams audio from Microsoft’s Edge read-aloud service over the network — it’s free and keyless, but not offline like Whisper/Qwen.
5. **share=True + debug=True in production-style use.** Fine for a demo, but worth flagging that this creates a publicly accessible URL and leaves debug tracebacks visible.
6. **Fixed retrieval depth (k=5) regardless of query.** No re-ranking or relevance thresholding, so low-similarity memories can still get injected into the prompt on topically narrow queries.
7. **1.5B model capacity.** Reasoning quality, instruction-following, and long-context coherence will be materially weaker than GPT-4o — appropriate for pipeline validation, not for evaluating “memory quality” in isolation, since generation quality will confound results.

6. Suggested Next Steps

- Add a `user_id`-scoped collection or metadata filter to make the memory layer multi-user safe.
 - Add a lightweight relevance threshold (e.g., cosine similarity cutoff) before injecting retrieved memories, to avoid noisy context on unrelated queries.
 - Wrap `agent.chat`, `transcribe`, and `synthesize` in try/except with user-facing error messages in the Gradio callbacks.
 - Consider swapping `Qwen2.5-1.5B-Instruct` for `Qwen2.5-7B-Instruct` (4-bit quantized) — consistent with the model you already used successfully in your RAG agent project — to get materially better reasoning while still fitting a T4.
 - Add memory summarization/compaction (e.g., periodically collapsing old exchanges into a condensed summary document) to keep retrieval quality stable as the collection grows.
-

7. Summary

This notebook is a well-structured, minimal-dependency reference implementation of a memory-augmented conversational agent, built entirely on free/local tooling. Its main value is as a clean architectural template — model, memory, agent orchestration, voice I/O, and UI are cleanly decoupled — rather than as a high-capability assistant. The most impactful next steps are multi-user memory isolation, retrieval quality controls, and basic error handling, all of which are straightforward additions to the existing structure.